Latest updates: https://dl.acm.org/doi/10.5555/781915.781933

PDF Download
781915.781933.pdf
05 January 2026
Total Citations: 1
Total Downloads: 597

Published: 07 November 1995

Citation in BibTeX format

ARTICLE

Software architectural analysis: an experience report

MAURICIO DE SIMONE, University of Waterloo, Waterloo, ON, Canada

RICK KAZMAN, University of Waterloo, Waterloo, ON, Canada

Open Access Support provided by:

University of Waterloo

CASCON '95: Proceedings of the 1995 conference of the Centre for Advanced Studies on Collaborative research (November 1995)
https://doi.org/10.5555/781915.781933

Software Architectural Analysis: An Experience Report

Mauricio De Simone

Department of Electrical and Computer Engineering
University of Waterloo, Waterloo
Ontario, Canada, N2L 3G1
mdesimon@etude.uwaterloo.ca

Rick Kazman

Department of Computer Science
University of Waterloo, Waterloo
Ontario, Canada, N2L 3G1
rnkazman@watcgl.uwaterloo.ca

Abstract

As the software marketplace becomes more competitive, customers and developers demand more from their products. Software quality is now seldom measured solely in terms of executable features. Non-functional qualities, such as portability, modifiability and extendibility are becoming increasingly important in the success of a software product.

Software architecture analysis is based upon the assumption that the product's architecture is responsible for a substantial amount of its non-functional qualities. Making such analyses is important because they represent a relatively low-cost means of improving a product early in the software development life cycle or in determining directions for product evolution later in it. Moreover, if architectural analysis is done at the design stage and maintained at every stage of the development of the product, then it also constitutes a very effective tool for communication within the development team.

This paper presents the experience of applying SAAM (Software Architecture Analysis Method) to the analysis and evaluation of a commercial revision control system. The methodology is described, concentrating on (1) the usage scenarios of interest to the various users, developers, and administrators of the system; (2) a technique for characterizing the system at an architectural level; and (3) the evaluation and recommendations made, and their impact on the software development company.

1 Software Architecture

Software quality is seldom measured solely in term of executable functions. It invariably includes non-functional qualities such as portability, extendibility and modifiability. This demand for non-functional qualities comes from many sources, among them: software systems are getting larger and more complex; software development companies are attempting to develop product *families* rather than just single

products; networks of heterogeneous machines and protocols are now common, the complexities of new problems require that software tools be integrated in a more efficient way. This not only has impact on the use of a software product but also imposes new constraints on the software development process. Software manufacturers can no longer assume that customers' needs are going to be static or that their programs are going to operate in isolation.

One of the claims of the Software Architecture research community is that the architectural design of a product has a direct impact on the achievement of non-functional qualities [5], [6]. The layout of modules, functional decomposition, interfaces and protocols, isolation of functionality, etc., all play a major role in determining the ability of the product to achieve specified non-functional requirements. Furthermore, an accurate and detailed architectural representation will also aid in the development of complex systems because it provides a uniform, shared notation that can enhance the communication amongst the development team.

The problem of evaluating software architectures with respect to their non-functional qualities is complex for many reasons:

- There is no unit of measurement for non-functional qualities. We require a way of evaluating these qualities in an objective manner.

- There is currently no common vocabulary for architectural representation or architectural claims [7]. This is problematic because the semantics of each author, when presenting their architecture (meaning of a box, meaning of an arrow) are often different. Thus, there is a need for a common vocabulary as well as common measuring sticks.

- Most of the non-functional requirements refer to requirements which will be imposed in the future. The prediction of future modifications, for example, is a very complex task.

- Since the process of evaluation is based on forecasting (some might say fortune-telling), each author interprets non-functional qualities in their own style, with no clear way of comparing different author claims. For example, for an operating system designer portability might mean going from one hardware platform to another or from one disk drive controller to another. For an application designer, portability might mean going from one operating system or user interface toolkit to another.

- Lack of complete definition of non-functional features. For example, do we say that a system that can work on a different hardware platform by simply being recompiled, but requires dozens of changes to accommodate a different data type, is a modifiable system [1]?

For a solution to the problem of evaluation we use SAAM: the Software Architecture Analysis Method [7], [8]. SAAM carries out context-based evaluation through the use of task scenarios. These scenarios describe specific tasks that will be carried out by various people involved with the system: users, developers, maintainers, administrators, etc.

The work described in this paper is the application of SAAM to a commercially available revision control system, which we will call WRCS. WRCS provides complex functionality to allow developers of projects the ability to create archives, compare files, check files in and out, create releases, back up to old versions of files, and so on.

We present here the steps taken to arrive at an architectural evaluation of WRCS and the evaluation itself. We conclude with recommendations and lessons learned. This is an instructive example because it is based on a real product, facing real-world difficulties and requirements (such as time and budget constraints). The complete process of this evaluation will also highlight the importance of an architectural description as a medium for communication.

2 SAAM: Software Architecture Analysis Method

There is growing interest in both the academic and industrial communities for the evaluation of non-functional properties of software products. Most software vendors and authors make claims about non-functional properties of their products but seldom provide adequate basis for their claims. Worse still, it is typically not even clear what an author means by their claims of maintainability or interoperability.

SAAM is based on examining architectural descriptions of a system. An architectural description contains information about what functions or services the system provides and how that functionality is allocated to the structure of interacting components which comprise the system. The evaluation portion of the method consists of determining how the well allocation of function onto a system's structure supports the activity of a particular scenario [1].

2.1 Overview of the Method

We will now briefly explain SAAM. For more detailed explanations of the method, see [1] or [7].

1. **Describe Candidate Architectures**: Describe the architecture or architectures to be evaluated in a common syntactic (i.e. structural) notation, preferably one that is shared within an organization. The structural representation of each architecture will show the systems components (computation and data) and their relationships (data and control). It is not necessary to describe the architecture in a formal notation. Any notation that provides a complete and accurate representation will suffice[1].

2. **Develop Scenarios**: Develop task scenarios that will illustrate what activities the system will be required to support. It is not necessary to have a clear, *a priori* understanding of which activities affect which non-functional qualities. This mapping from proposed task to non-functional qualities will become clear later in the evaluation. It is important to obtain tasks from every different type of interaction to the system. (i.e. developer, maintainer, user, etc.)

3. **Perform Scenario Evaluations**: For each scenario determine if the task can be carried out directly (by executing the system) or indirectly (by modifying the system). If a task is supported indirectly, determine the number of components and relationships that must change. The fewer the changes the better.

4. **Reveal Scenario Interactions**: Map all indirect scenarios onto the components and connections which they affect. Doing so will reveal those tasks which impose modifications to the same components of the system. This will give insight on the architecture's support for separation of concerns. The less scenario interaction for distinct classes of scenarios the better. For example, if scenarios 1 and 2 are of different classes and both affect component X, then component X is a potential trouble spot. The interaction of scenarios 1 and 2 illustrates that this architecture has not properly separated concerns. If, on the other hand, scenarios 1 and 2 are of the same class (say, change the font and change the colour of menus items, respectively) then their interaction within component X is illustrative of appropriate cohesion within the module.

5. **Overall Evaluation**: Finally, weight each scenario and scenario interaction as appropriate for the organization. This weighting will reflect the priorities of the organization within the objectives expressed in the scenarios. As a consequence, the same product will have a different evaluation for organizations with different goals. For example, some organizations might stress security above all else, whereas others might stress fault tolerance, etc.

[1] As complete and accurate as the analysis requires

This method provides a way of evaluating a product in a way that the results are natural to understand, are easy to explain, and are related to each role objectives within an organization's priorities. Developer, Maintainers, Users and Managers will find answers to their direct questions, so long as those questions are manifested in terms of scenarios.

This method was originally intended for the comparison of different architectural designs for the same problem. Within the context of this report we are going to use it for evaluating a single product. The method still applies, the only difference is that the results will have different meanings. When one is comparing, one can say product A is better than B in certain areas, or vice versa. In this report our evaluation will show which tasks are supported directly and which are not. This will give us insight to the non-functional capabilities of the product. It will also show weak and strong points of the product's architecture relative to the organization's objectives, via an analysis of scenario interactions.

SAAM has been proven successful in various case studies. It provided an evaluation of User Interface Management Systems in [7] and a evaluation of Internet Information Systems in [1]. In both cases the task scenarios used reflected the interests of a domain expert who guided the evaluation team towards the required capabilities of the product in those specific fields. This role is very important to the method, since the evaluations will be different if the scenarios imposed upon the system are different.

3 Case Study

3.1 Overview of the Product

The WRCS Revision Control System is a software product designed to aid in the management and organization of files in a computer-based development project. "Project" in this context means any group of related files that, when linked together appropriately, form a finished product, for example, source code for a computer program, text for creating a book, or digitized audio and video for the creation of a digital video clip. WRCS keeps track of changes made to files as they evolve over time. It provides capabilities for multiple users to work on the same project in their own working space, allowing each developer to modify and test the system in isolation, without disturbing other developers' work and without corrupting the primary copy of the system. Managerial functions, such as production of reports, are also provided.

3.2 Architectural Description

For any evaluation to take place we require an architectural representation of the product with a well-specified semantic interpretation (principally what it means to be a component or a connector). This proved to be one of the most difficult tasks of this evaluation. At the start of this project there was *no* architectural description of WRCS, therefore we had to devise a way of eliciting this information.

This information had to be analyzed and grouped in a way that would aid in the construction of an architectural diagram. Our sources of information were limited; they consisted of interviews with some of the members of the development team, the documentation of the product, and the product itself. We had no access to the source code or the specifications of the product; this is appropriate in that software architecture is supposed to concern itself with a level of abstraction above code. In essence, our task was to create a design document out of a finished product: reverse engineering.

The description of the product was arrived at in an iterative way, in stages, with each stage being the starting point of the next one. At each stage we studied the description (and all other sources of system information at our disposal) as it existed and devised a new set of questions. The answers to the questions in each stage helped us to clarify the current description. Each new stage allowed us to obtain more insight on the product and motivate new questions to be asked in order to arrive at the next stage (Fig. 1). Since we did not have any previous representation, we chose to start with a gross listing of the modules along with their basic relationships and from there iterate. It took three iterations to obtain a representation good enough for our purposes.

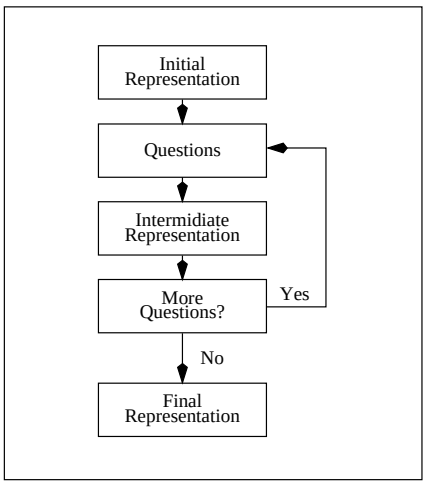

Figure 1: Iterative Approach

A simplified version of the architectural diagram obtained is presented in Fig. 2.

3.3 Scenarios

Once we arrived at an architectural description of the product, we need to enumerate tasks that will be required from the system, as described in Section 2.1.

The tasks that we present here are a subset of the tasks which were elicited from the domain expert. In total we studied 15 tasks (6 classified as user tasks, 4 as developers tasks, 5 as administrator); here we present 6:

1. **Understand other products' binary file representation.** Give meaning to binary files generated by other products, for example the visual difference of WordPerfect[2] document should not be viewed as a difference in binary files but as a difference in WordPerfect documents.

2. **Configure the product's toolbar.** Change the icons and actions associated with a button in the toolbar.

3. **Port to another operating system.** Go from Microsoft Windows[3] to for example IBM OS/2.[4]

4. **Make minor modifications to the user interface.** Add a menu item, change the look and feel of a dialog box.

5. **Port to a different network management system within the same operating system.** Change from Novell's NetWare[5] to, for example, Microsoft Lantastic[6].

6. **Integrate with a new development environment.** Attach for example to Symantec C++[7].

Tasks 1 and 2 are user proposed task; 3 and 4 are developer, 5 and 6 are administrator tasks.

3.4 Evaluation

The evaluation commences with simply determining if the proposed tasks are supported directly or indirectly. If tasks are supported indirectly then we indicate which modules must be modified.

[2] WordPerfect is a trademark of WordPerfect Inc.
[3] Windows is a trademark of Microsoft Corp.
[4] OS/2 is a trademark of International Business Machines Corp.
[5] NetWare is a trademark of Novell Corp.
[6] Lantastic is a trademark of Microsoft Corp.
[7] Symantec C++ is a trademark of Symantec Corp.

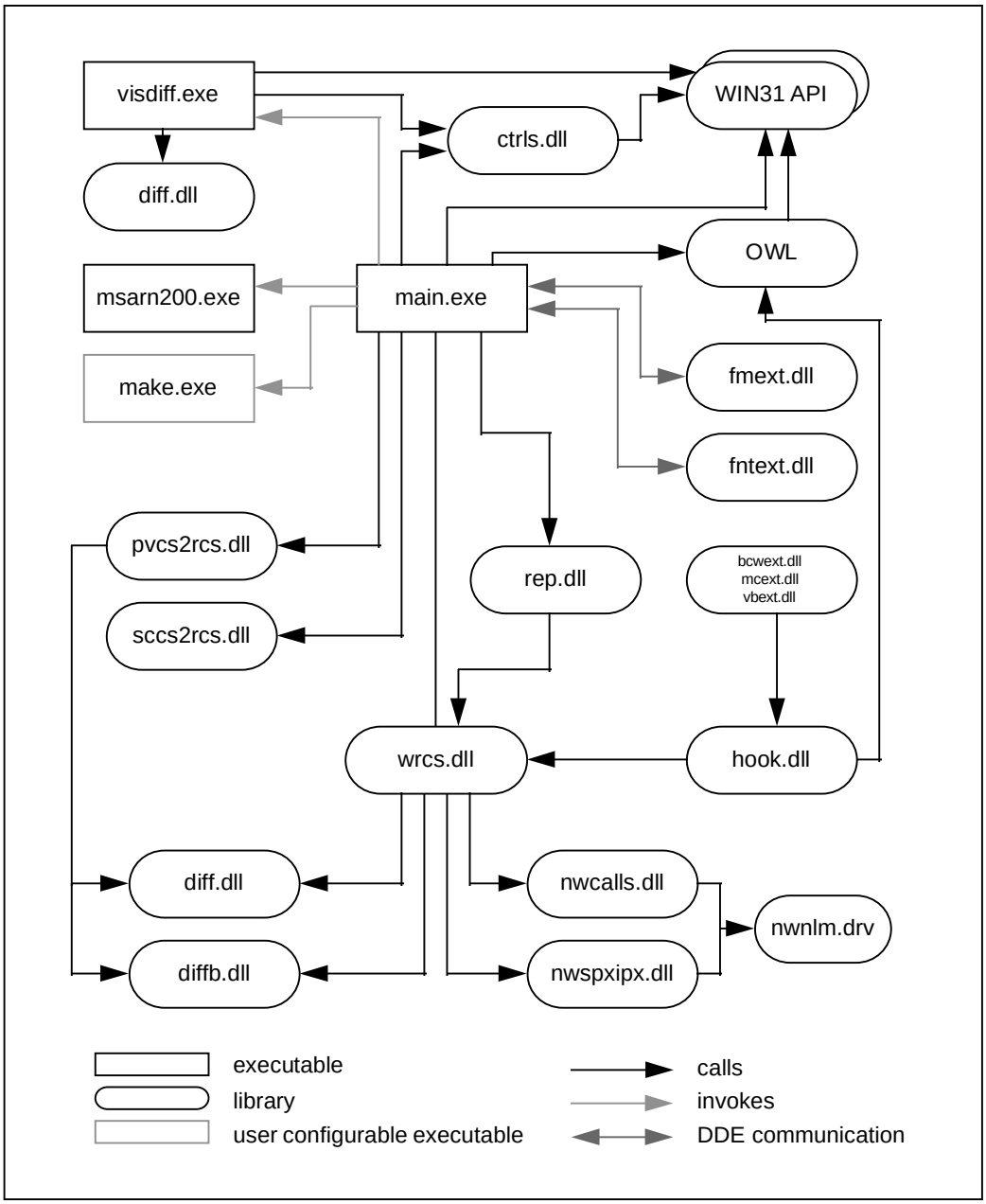

Figure 2: Architectural Diagram

1. **Understand other products binary file representation**. *Indirect Task.* This will require changes in diffb.dll. If this new feature is to be shown visually it will also require changes in visdiff.exe.

2. **Configure the product's toolbar**. *Direct Task.*

3. **Port to another operating system**. *Indirect Task.* All components that call the Window's 3.1 Aplication Programmer Interface (WIN31 API) would have to be modified: main.exe, visdiff.exe, ctls.dll, OWL. Borland's ObjectWindows Library (OWL)[8] has been ported to other operating systems, if our target platform is supported by OWL no modifications are needed there. The concept of a Dynamic Link Library might not be supported in the new operating system. Changes would be needed appropriately. wrcs.dll would have to be modified to take into account the new networking environment. The invocation mechanisms present in main.exe would have to be revised and changed. The menu extensors would have to be rewritten, and the inter-process communication between the extensors and hook.dll would have to revised. New modules for the integration to the new operating system file manager equivalent would have to be devised. The communication between this new modules and main.exe would have to be change if Dynamic Data Exchange (DDE) is not supported in the new operating environment.

4. **Make minor modifications to the user interface**. *Indirect Task.* Requires modification of main.exe, visdiff.exe, and hook.dll. Possible modifications in ctls.dll will be required.

5. **Port to a different network management system within the same operating system**.*Indirect Task.* This will require changes in wrcs.dll.

6. **Attach to a new development environment**. *Indirect Task.* This is an indirect task, since is not supported within the current functionality of WRCS. Nevertheless the presence of hook.dll supports the addition of new menu extensors in a simple way.

[8] OWL and ObjectWindows are trademarks of Borland International Inc.

Module	# of Changes
main.exe	4
wrcs.dll	7
diff.dll	1
diffb.dll	1
pvcs2rcs.dll	1
sccs2rcs.dll	1
nwcalls.dll	1
nwspxipx.dll	1
nwnlm.drv	1
hook.dll	4
rep.dll	1
visdiff.exe	3
ctrls.dll	2

Table 1: Changes per module

3.5 Scenario Interaction

When two or more task scenarios require change in the same component of a system, they are said to interact. Scenario Interaction is a very important consideration because it exposes the allocation of functionality to the product's design. In a very explicit way it is capable of showing which modules of the system are involved in tasks of different nature. High scenario interaction reveals a poor isolation of functionality in a particular component of a design, giving a clear guideline on where to focus the designer's subsequent attention.

In our case we created a table (Table 1) that shows the number of changes required in each module of the system[9]. Since each task imposes only a single change, the number of changes per module indicates the level of scenario interactions for it.

Two of the main purposes of software architecture and of architectural analysis are as a communication tool within a software team and

[9] Here we are considering all the tasks presented to WRCS, not just the ones presented in this paper.

as a means of documenting a design and design rationale. As such, we must be very concerned with how we present the results of an architectural analysis, so that they are clearly transmitted, and so that they properly emphasize problem areas.

One visualization technique we have used to highlight scenario interactions is a fish-eye view. In Fig. 3 the architectural diagram of WRCS (Fig. 2) is presented with module size made proportional to the number of interacting scenarios which affect it. This figure clearly shows where the scenario interactions lie and the relative scale of the interactions. It can be seen clearly that the component with most scenario interaction is wrcs.dll and it is within this component that most of the future effort will be concentrated. Modules main.exe, visdiff.exe, hook.dll also suffer from high scenario interaction.

This information immediately calls attention to the most architecturally significant features of the system, as it currently exists, and guides designers and developers in their allocation of time and effort.

4 Recommendations

Now that we have performed this analysis, we can conclude that WRCS does not support the following non-functional attributes:

- Portability in terms of the user interface is clearly not supported. The large number of changes and the large number of places affected by a port to a new interface toolkit indicate no architectural support for this quality.

- Modifiability in terms of user interface modifications is also not supported, since the development was tied very closely to the WIN31 API, without providing an adequate abstraction layer.

- Modifiability in terms of the representation of the underlying archive format is also not supported. This is because the functionality which understands the archive format is scattered among three modules.

- Security. WRCS's architecture provide no support for the security of the underlying archive.

One non-functional attribute is, however, supported indirectly:

- Extendibility. It is fairly straightforward to add the functionality required to attach to existing development environments. The component hook.dll provides a gate to WRCS' functionality.

There are several common guidelines that will allow WRCS to overcome the difficulties discovered by SAAM's evaluation. The main problem in the current architecture is lack of proper separation of concerns. So any technique aimed at properly defining separation of functionality will aid in improving the product. A substantial portion of WRCS's code is devoted to its graphical user interface and most of its difficulties in managing portability and modifiability are concentrated in the graphical user interface portion of the program. Several solutions to the problem of appropriately separating concerns in interface-dominated applications currently exist ([2], [3], [4]). None of these, however, would provide a solution for WRCS, as we will discuss next.

5 Lessons Learned

This evaluation confirmed our belief that architectural analysis is a very inexpensive way to evaluate and improve the design of software products. None of the steps of this analysis depend on a particular implementation language, style or methodology. Thus, architectural analysis can be applied before the system is designed in detail and before any code is implemented, and the analysis will be as effective. For our particular case this analysis proves that WRCS is a product with severe limitations in achieving non-functional requirements and a major redesign/rewrite seems appropriate. Going through an analysis procedure such as SAAM before implementation would have substantially contributed to avoid the problems which the WRCS developers now face.

Within the organization this evaluation obtained mixed results. Senior developers and

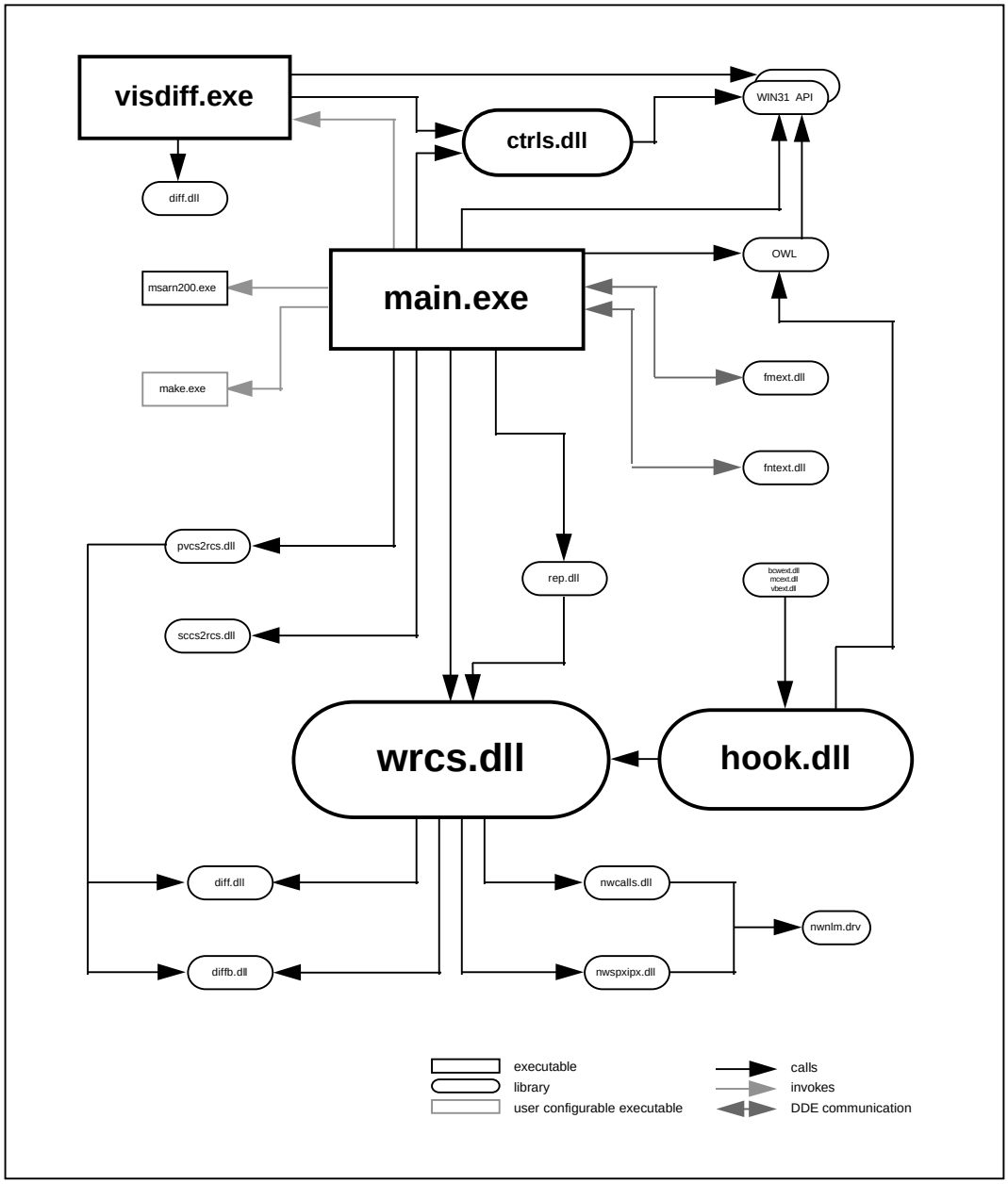

Figure 3: Fish-eye Visualization of Scenario Interactions

managers found a very important tool in architectural analysis and plan to impose it in future developments of new products. Within the WRCS team, however, this evaluation was regarded as just an academic exercise. We attribute this to the fact that senior developers and managers have enough perspective to understand that the majority of the software development life cycle is spent in maintenance and feature enhancements. For this reason, any effort that will aid in improving a product's support for non-functional qualities, such as maintainability or modifiability, is important. However, the developers within the WRCS team do not have this broad perspective. When one is concerned with meeting the next release deadlines or with finding a bug, there is no time for the luxury of contemplating major changes to the architecture. In the words of one senior manager: "they have features to implement". This is why architectural analysis must be done early. Otherwise, it will never be done or, if done, it will be meaningless.

It should be stressed that since architectural evaluation is so easy to carry out and can be done at a very early stage in the life cycle, it will not seriously compromise the organization's time and resources.

SAAM allowed an insight to the product capabilities that could not be easily achieved from inspections of code and design documents. In a very simple, straightforward and inexpensive way it exposed specific limitations of a product. Furthermore, this was accomplished with only scant knowledge of the internal workings of WRCS. We feel that these are significant achievements.

About the Authors

Mauricio De Simone is a Master of Science candidate at the Department of Electrical and Computer Engineering at the University of Waterloo. His primary research interests include software engineering (software architecture), parallel and distributed systems (development of flexible environment for the creation of distributed systems), human-computer interaction (graphical user interfaces) and programming languages. De Simone received his Electronic Engineer degree from the Universidad Simon Bolivar, Caracas, Venezuela.

Rick Kazman is an Assistant Professor in the Department of Computer Science at the University of Waterloo and Visiting Scientist at the Software Engineering Institute, Carnegie Mellon University. His primary research interests are software engineering (software architecture and program visualization), multimedia information systems, human-computer interaction (particularly interaction with 3D environments), and computational linguistics (information retrieval). Kazman received a B.A. and M.Math from the University of Waterloo, a M.A. from York University and a Ph.D. from Carnegie Mellon University. He is a member of the IEEE Computer Society and the ACM.

References

[1] G. Abowd, R. Kazman, L. Bass, and J. Pitkow. "Scenario-based evaluation of software Architectures: A case study on Internet information systems". *Submitted to SIGSOFT'95*, 1995.

[2] J. Coutaz. "PAC, an Object Oriented Model for Dialog Design". *in Human-Computer Interaction H.-J. Bullinger and B. Shackel (Eds.) Elsevier Science Publishers B.V.*, pages 431 – 436, 1987.

[3] J. Coutaz. "Architectural Design for User Interfaces". *ESEC*, 1991.

[4] J. Coutaz. "A Metamodel for the Runtime Architecture of an Interactive System". *The UIMS Tool Developers Workshop. SIGCHI Bulletin, Vol. 24*, pages 32–37, 1992.

[5] D. Garlan and M. Shaw. "An Introduction to Software Architecture". *Advances in Software Engineering and Knowledge Engineering. World Scientific Publishing Company*, I, 1993.

[6] R. Kazman and L. Bass. "Toward Deriving Software Architectures from Quality Attributes". *Carnegie Mellon University, Software Engineering Institute Technical Report CMU/SEI-94-TR-10*, 1994.

[7] R. Kazman, L. Bass, G. Abowd, and M. Webb. "SAAM: A Method for Analyzing the Properties of Software Architectures". *Proceedings of the 16th International Conference on Software Engineering, Sorrento, Italy*, pages 81–90, 1994.

[8] R. Kazman P. Clements, L. Bass and G. Abowd. "Predicting Software Quality by Architecture-Level Evaluation". *Proceedings of the 5th International Conference on Software Quality, Austin, TX (to appear)*, 1995.